



Module 1: Data Pipeline (/data_pipeline)

An end-to-end automated ETL (Extract, Transform, Load) pipeline built for **Zepto Catalog & Competitive Intelligence Analytics**. This module extracts real-time product data, parses unstructured copy-pasted text snippets, cleans messy fields, normalizes currency into INR, loads the dataset into a normalized SQLite relational database, and executes analytical queries in both SQL and Pandas to verify data parity.

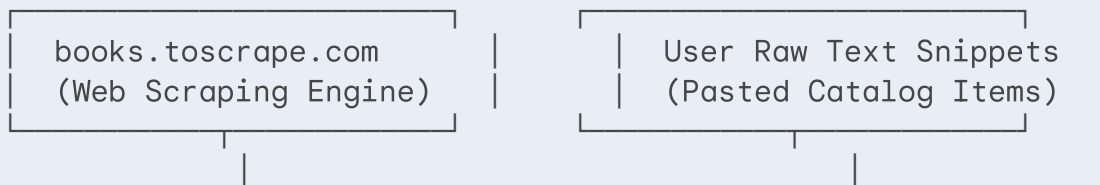


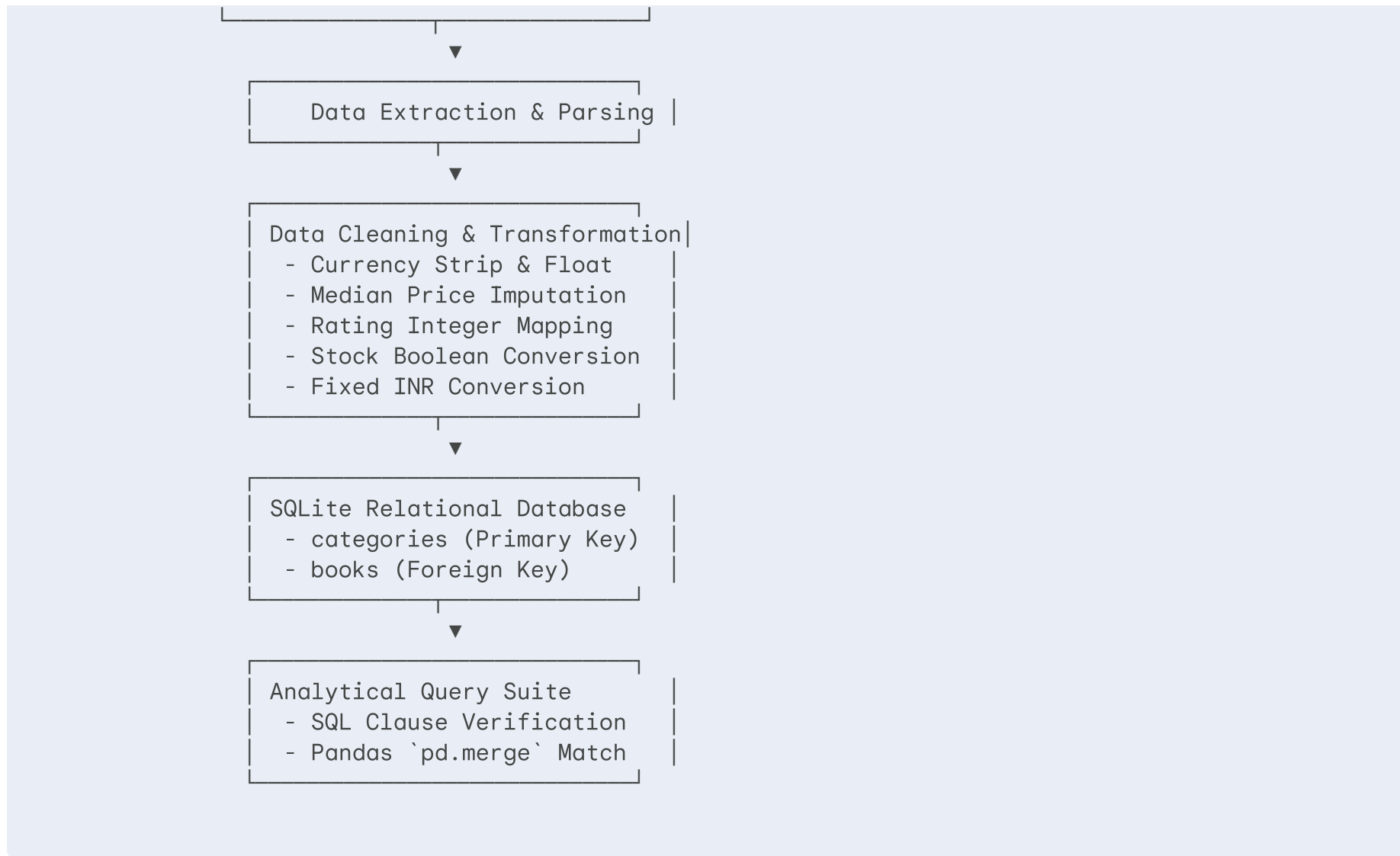
Table of Contents

- [1. Pipeline Overview & Data Flow](#)
- [2. Baseline Currency Exchange Rate](#)
- [3. Data Cleaning & Parsing Strategy](#)
- [4. Raw Text Snippet Parsing Support](#)
- [5. Database Schema \(SQLite\)](#)
- [6. SQL Query Suite & Clause Coverage](#)
- [7. SQL vs. Pandas Equivalence Verification](#)
- [8. Installation & Setup Guide](#)
- [9. Project Structure](#)



Pipeline Overview & Data Flow





Baseline Currency Exchange Rate

- **Fixed Baseline Rate:** 1 GBP = 105.50 INR
- **Specification:** Project-defined static baseline constant. No live network API or date lookup required during core execution.



Data Cleaning & Parsing Strategy

Field	Raw Source Format	Cleaned Target Format	Rationale & Cleaning Strategy
title	HTML title tag or raw text line	TEXT / string	Extracted via BeautifulSoup <code>h3 a[title]</code> or regex line extraction. Rows missing a title are dropped directly.
price_gbp	String (e.g., £51.77 , £16.85)	REAL / float	Sanitized using regex <code>[^\d.]</code> to strip currency symbols. Missing or broken values use median price imputation.
rating	Text class (e.g., star-rating Three)	INTEGER (1–5)	Mapped from English words (One .. Five) to integers 1–5 via dictionary lookup. Missing ratings use median imputation.
in_stock	Text string (e.g., In stock)	INTEGER (0 or 1)	Case-insensitive check for "in stock" . Assigns 1 if available, 0 otherwise.
price_inr	Derived value	REAL / float	Computed as $\text{price_gbp} \times 105.50$ rounded to 2 decimal places.



Raw Text Snippet Parsing Support

The pipeline features a dedicated raw text parser (`parse_pasted_text_results`) that ingests unstructured copy-pasted web snippets directly.

Key Features of Text Parser:

- Title & Price Matching:** Uses regular expressions (`£\d+\.\d{2}`) to dynamically associate book titles with price lines.
- Automated Category Mapping:** Matches title keywords (*Himalayas, Europe, Provence, Railway, Tuscan, Road, Places, etc.*) to dynamically categorize books into **Travel** or **Biography**.

- **Pipeline Integration:** Cleaned snippet records are seamlessly merged with web-scraped entries prior to database storage.

Database Schema (SQLite)

The SQLite database (`catalog_benchmark.db`) implements a 3NF normalized schema with strict Foreign Key enforcement (`PRAGMA foreign_keys = ON;`).

1. `categories` Table

```
CREATE TABLE categories (  
    category_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    category_name TEXT UNIQUE NOT NULL  
);
```

2. `books` Table

```
CREATE TABLE books (  
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    price_gbp REAL NOT NULL,  
    price_inr REAL NOT NULL,  
    rating INTEGER NOT NULL,  
    in_stock INTEGER NOT NULL,  
    category_id INTEGER NOT NULL,  
    FOREIGN KEY (category_id) REFERENCES categories (category_id)  
);
```



SQL Query Suite & Clause Coverage

The pipeline executes 5 benchmark SQL queries covering all required relational SQL operators:

1. **SELECT / WHERE + ORDER BY:** Filters available high-value books (`price_gbp > 30.0`) ordered by price descending.
2. **DISTINCT + LIMIT:** Retrieves the top 10 distinct star rating and price combinations.
3. **IN Clause:** Filters books matching specific category names (`'Travel'` , `'Sequential Art'` , `'Mystery'` , `'Music'` , `'Biography'`).
4. **BETWEEN Clause:** Filters books priced in the mid-tier INR range (`2000.0` to `4000.0`).
5. **JOIN Clause:** Joins `books` and `categories` tables to create a complete catalog summary.



SQL vs. Pandas Equivalence Verification

To verify pipeline consistency, Query 5 (relational JOIN) is executed via two parallel pathways:

1. **SQL Method:** `pd.read_sql_query()` executing an explicit SQL `INNER JOIN` .
2. **Pandas Method:** In-memory `pd.merge()` combining `books_df` and `categories_df` on `category_id` .

```
# Pure Pandas Verification Logic
pandas_merged_df = pd.merge(books_df, categories_df, on="category_id", how="inner")
assert sql_join_df.equals(pandas_merged_df), "Mismatch between SQL JOIN and Pandas merge!"
```

Result: Both approaches yield 100% identical DataFrame outputs.



Installation & Setup Guide

Prerequisites

- Python 3.8 or higher

- `pip` package manager

1. Installation

```
pip install requests beautifulsoup4 pandas numpy
```

2. Execution

Run the complete pipeline from the project root or `/data_pipeline` directory:

```
python data_pipeline/pipeline.py
```

Directory Structure

```
data_pipeline/  
├─ pipeline.py          # Main ETL pipeline script (Scraping, Cleaning, DB & Queries)  
├─ catalog_benchmark.db # SQLite database generated at runtime  
└─ README.md           # Standard module README documentation
```